Zhenyu Jiang
A18098929

Demo link: https://youtu.be/J40sx3TA0G4
Github repo: https://github.com/XDawn66/Almost-barrer-control/tree/main

The implementation of the Almost Barrier Function

Intro:

The definition of the Barrier function is that given a dynamic system x = f in state space R^n, and let X_s be the safe region (or safe state set), and X_u be the unsafe region (or unsafe state set. $X_S \cup X_U = X$, and $X_S \cap X_U = 0$ There exist a function B that it satisfied the following: $B(x) \geq 0, \; in\; all\; x \in X_s$, $B(x) \leq 0, \; in\; all\; x \in X_U$, and $L_f B(x) > 0, \; where\; B(x) = 0$. The $L_f B(x)$ is the Lie Derivatives of the system, which is continuously differentiable, and the purpose of this derivative is to see how the direction of the system dynamic changes over time. In the general case, we want it to always point to the safe zone and bounce off from the safety boundary. However, the barrier function is only providing very boolean results; it's not very useful when we are dealing with a complex, high-nonlinear system. The paper proposes the almost barrier function, which is a less restricted version of the regular barrier function. As a result, it is able to provide more informative information about action states by using invariant-based analysis, and it is able to certify safety for the majority of the state space.

Set up:

I define my system as a simple Kinematic model where

$x = vcos(\Theta)\; y = vsin(\Theta)\; \Theta = v\, tan(\delta)/L\; v = a$

The set up for the simulator is inspired by:
https://github.com/NeuralNine/ai-car-simulation

https://www.youtube.com/watch?v=Cy155O5R1Oo

Instead of using their NEAT (NeuroEvolution of Augmenting Topologies) for Reinforcement learning, I used the same technique as the paper:
Quantifying Safety of Learning-based Self-Driving Control Using Almost-Barrier Functions
With SAC(Soft Actor Critic) which is setted up in the gymnasium(Gym) with the custom Env file. On the other hand, I used sklearn for the KNN, pickle for data collection, and pytorch for neural network training.

The 2D car has 2 sensors, each of them is shooting to the 45 degree angle to the front-left and front-right. The speed of the car is set to 10, with the size of 60 by 60 in pygame. It can speed up or down based on the control, and it can steer left or right.

Step 1: Training the control policy

In the paper, the author uses a reward function that awards goal reaching and penalizes the angle error and the distance error from the center. However, in our case, it's difficult for us to achieve the same reward function due to not knowing the coordinate of the track boundary. Furthermore, the "correct" direction is not fixed in our case, and facing from the starting direction is not necessarily able to finish the track. Our track is a road with sharp corners and eventually loops back to the starting point. As a result, I need to come up with different ways to design the effect reward function to allow the car to finish as many different labs as possible. After spending a month of trying and trained 60 + MLP models, I conclude the following pattern:

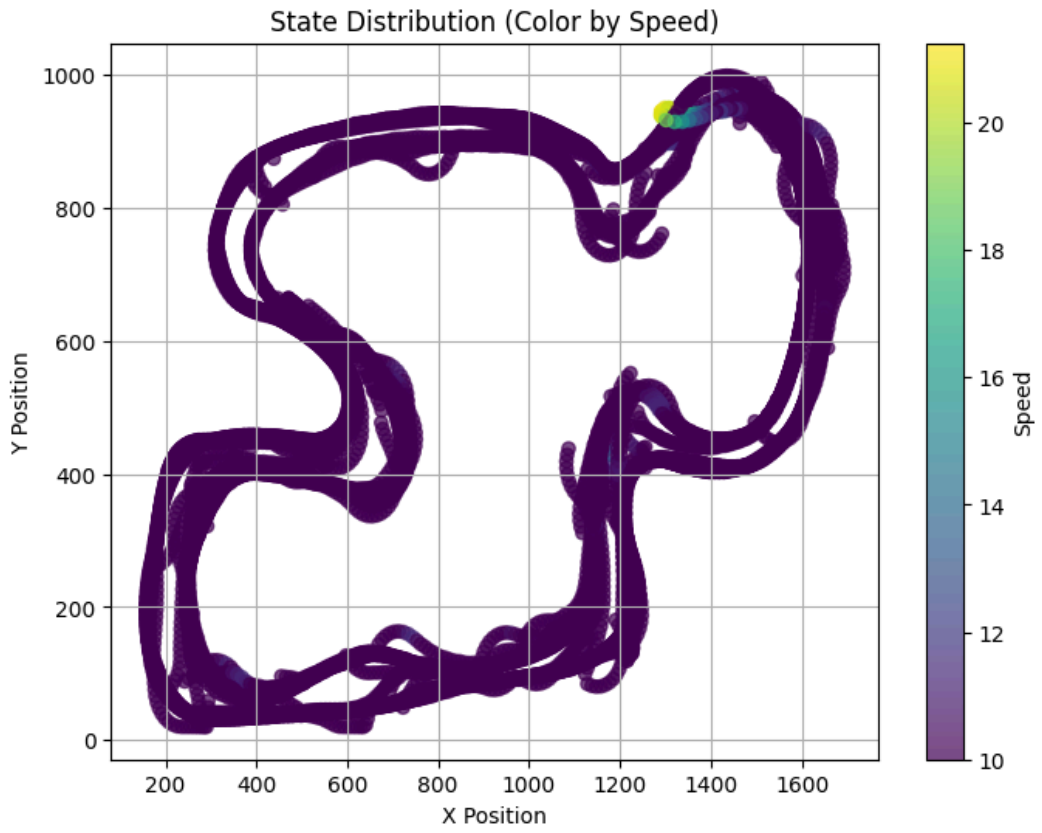
- Overly penalize(big negative reward in the end) the car will discourage the agent to move forward and crash early
- Penalizing having sharp turns is not effect in this case
- Penalizing going too fast is not very effective in this case
- Rewarding the goal reaching is not very effective at the early training process, since the car don't even have the chance to reach the goal before it crash in most of the time
- Penalizing the car not staying at the center is helpful to finish the track

- Penalizing crashing early in the decent value is helpful to guide the car to finish the track
- Rewarding the car as it go further is significantly helpful to guide the car to finish the track
- Rewarding the car staying alive is helpful to guide the car to finish the track
- Rewarding the car as it reach the checkpoint is significantly helpful to guide the car to finish the track, meaning letting the car knowing it is closer to the goal is essential to guide the car to finish

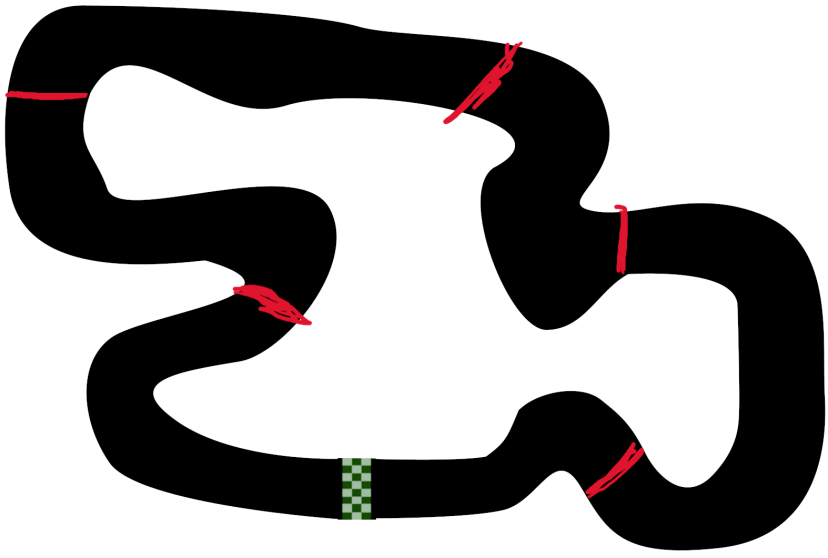
I initially didn't realize training the control policy was going to be such a difficult task, and I spent months training it in order to pass all the maps I have. In the meantime, I realize the fact that more timestep doesn't mean the control will be trained better, designing a good reward function is effective and essential for the car to finish the track.
In the end, I choose to use: Penalizing the car not staying at the center (corresponding to the angle error idea), Rewarding the car staying alive, Rewarding the car as it reach the checkpoint and the further it goes (corresponding to the goal reaching idea), Penalizing the car not staying at the center, and Penalizing crashing in the decent value as my final reward function.
My setup has 5 maps, starting with the easiest all the way to the most difficult one.
Unfortunately, even though I try my best to adjust the rewards/penalty, my design can only allow me to come up with a control that can pass the first 4 maps. I guess I need to learn more techniques to design an effect reward function so that I can come up with something that can finish all 5 tracks. The best model I came up is called "models/MlpPolicy_try_6"

Step 2: Collecting the safe data

This part I follow the same implementation as the paper, and in my case, I used my median difficult map(map 2) to collect 20k data points with my best model. During the data collection, the program randomly puts the car in a valid area, which means inside the track, and then starts collecting data until it crashes or finishes the current loop. When I finished the data collection, I was able to obtain the safe data set as shown below:



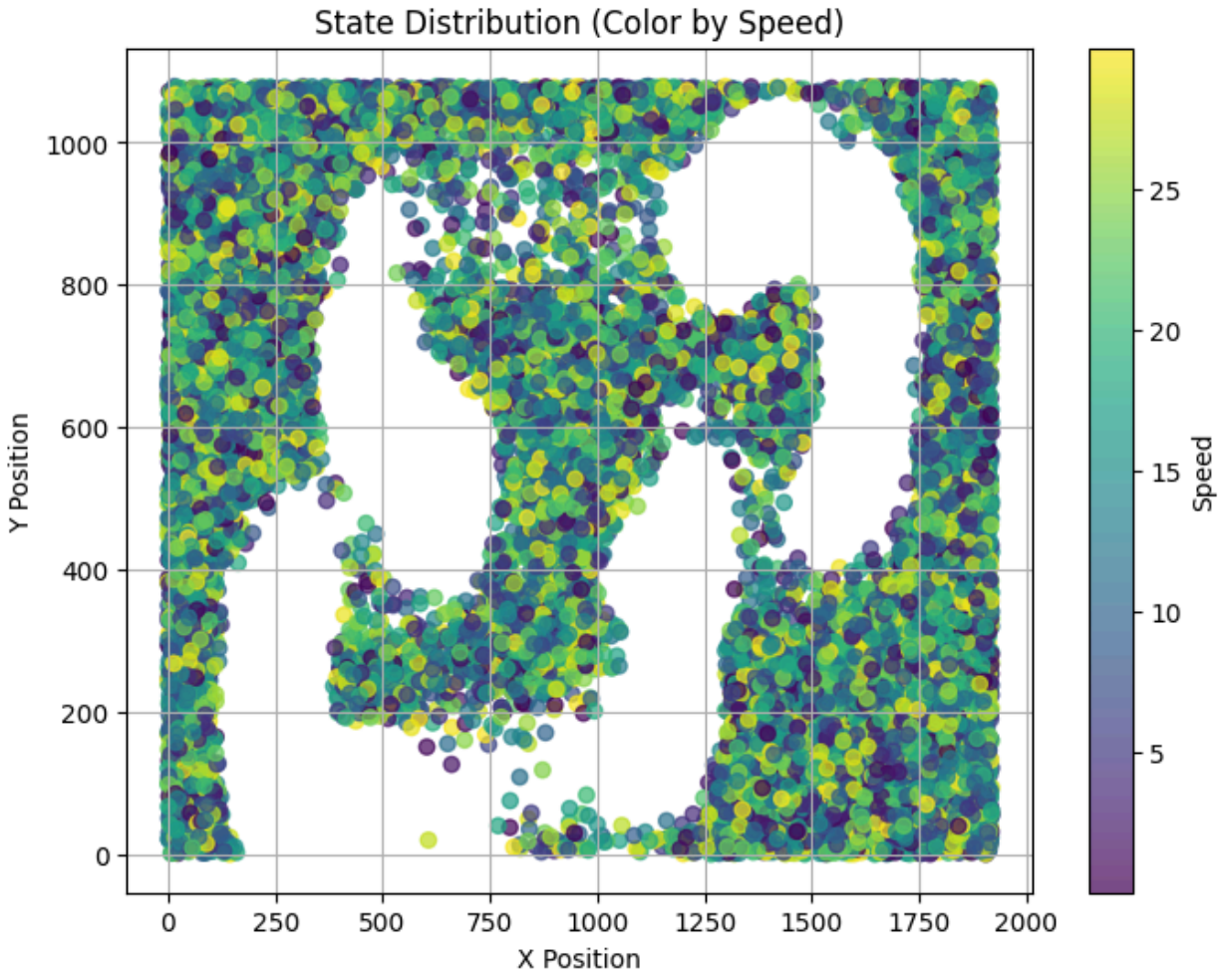
The original map:



Although the data couldn't represent the complete boundary of the map, I would say my safe data is able to capture the basic shape and boundary for the safe area.

Step 3: Compute the unsafe data

For this part, I also follow what the paper did. I sample M observations in the state space at each iteration, and then use kNN to label all the temporary candidate set X_c. If the majority of the neighbors are from the safe set, I will remove x_c from the X_c, otherwise it will be append to the unsafe set X_U . In my case, I set M to be 4000 and the number of neighbors to check to 18 in order to get a reasonable outcome. Again, I collect 20k in total for the unsafe set, and here is what the unsafe set looks like in the end.



Although the data couldn't represent the complete unsafe area of the map, I would say my unsafe data is able to capture the majority of the unsafe area.

Step 4 Train Barrier Function:

As we got the safe and unsafe dataset, we can now start the train barrier function. I use pytorch to build a single layer with tanh activation, following the equation

$B(x) = W_1^{(2)} \sigma'(W^{(1)}x + b^{(1)}) + b_1^{(2)}$

And use the lost function:

$$L(\Theta) = w_s 1/N_s \sum_{i=0}^{N_s} \phi(-B(x_s^i)) + w_u 1/N_u \sum_{i=0}^{N_u} \phi(B(x_u^i)) + w_l \sum_{i=0}^{N_s} \phi(-LB(x_s^i) - \gamma B(x_s^i))$$

I set $w_s = 1, w_u = 1, w_l = 1, and\, \gamma = 1$ in my case. I tried setting

$w_s = 1.5, w_u = 1.5, w_l = 0.5, and\, \gamma = 1$, but I couldn't find a reasonable Barrier Function In that case. In the end, $w_s = 1, w_u = 1, w_l = 1, and\, \gamma = 1$ works the best for my dataset.

In order to find the Lie Derivative, I use the approximation

$L_{f, theta\, t} B(s) = (B(s') - B(s))/theta\, t$, where theta t is set to 0.1 s.

s' is the next state of the current state s, and s' is obtained by reading the next trajectory from the safe dataset compared to the current trajectory.

After training for 100k step, the Barrier Function converge to the lost of ≤ 0.001

Step 5 & 6 Find dynamic on Boundary & verify barrier function lie derivative on Boundary

The idea of finding the dynamic on the boundary is first to create grids around the boundary, and then estimate the Jocobian of the dynamic in the local neighborhood. We sample the system's dynamics at states that are δ away from the grid centers in each dimension. This approximation gives an idea how the dynamics evolve in the grid cells. Delta is being set to 0.05 in my implementation. Unfortunately, I was not able to come up with a great implementation that works on my models, so I rely on LLM to help me finish this part.

For verify barrier function lie derivative, I follow the equation:

$L_f B([a]_\delta) = theta\, B([a]_\delta) \odot f([a]_\delta)$

Where f is the dynamic model that I created earlier with input of 0 to the controller.

Step 7 Retraining:

The author said that to improve the neural barrier function, we can retrain by adding the violating states during certification. Violating states are defined as: if

$B_U(x) > 0,\ B_L(x) < 0,\ and\ L_f B(x) < 0$

However, I couldn't get any violating states in the end, maybe the way I created my grids or the way I compute my lie derivative for my barrier function is incorrect. I tried to fix it but still couldn't figure out what the problem was.

A valid Barrial function must meet the 3 conditions as the definition:

$B(x) \geq 0,\ in\ all\ x \in X_s$, $B(x) \leq 0,\ in\ all\ x \in X_U$, and $L_f B(x) > 0,\ where\ B(x) = 0.$

After training 9 different models and all of their loss converges to a very small loss, I couldn't find a barrial function that can perfectly satisfy all 3 conditions. In the end, I was able to find a function that $B(x) \geq 0,\ in\ all\ x \in X_s$ and some $B(x)\ not\ \leq 0,\ in\ all\ x \in X_U$ and and $L_f B(x) > 0,\ where\ B(x) = 0.$

I also try to loosen the condition for the valid barrier function, given that allowing 1.5% of violation state for each condition. However, the violating rate for the barrier function is much higher than my previous approach.

Result:

For the neural controller that guide by the following reward function and rotate the training map in every 3k step:

- Penalizing the car not staying at the center (corresponding to the angle error idea)
- Penalizing the car not staying at the center
- Penalizing crashing in the decent value
- Rewarding the car staying alive
- Rewarding the car as it reach the checkpoint and the further it goes (corresponding to the goal reaching idea)

For the model: "models/MlpPolicy_try_6"

Map	Easy	Relatively easy	Regular	Relatively complex	Complex
Finish the track	Yes	Yes	Yes	Yes	No

For the natural burial function: “checkpoint 6.pth”

Map	Easy	Relatively easy(where the safe data set is collected)	Regular	Relatively complex	Complex
Certified rate($B(x) \geq 0$ /total number of B(x))	41.8%	90.9%	77.3%	82.3%	85%

Conclusion:

My implementation demonstrates one of the possible ways to create a neural controller for kinematic models and its almost barrier function. I encountered many challenges, such as not knowing how to start, struggling to get the perfect model, and finding the complete certified barrier function using a neural network. However, I was able to train a neural controller that finished most of the maps that I set up, and the barrier function I trained was able to have a relatively high certified rate. (average is around 80%). One fun fact that I observe while I am recording the certified rate is that the rate drops quite significantly as the simulation goes longer. Since the car is never crashing in the simplest setting, the simulation can go much longer than other maps. One assumption I have is that since my data only aim for recording the car finishing one track/loop, which is relatively short in most of the case, the barrier function marks more regions as less safe due to the unexpected length of the track. Overall, I gained vulnerable experience on training RL models from scratch and learned how to make a simple neural network with provided loss function. Hongzhan told me that this almost barrier function is powerful, but eventually, this is more like a way to certify a neural controller rather than an advanced self-driving controller. At first, I didn’t know what he was talking about, but things started to make sense as I was implementing. The barrier function is still very restricted since it’s not easy to find the barrier function that completely satisfies all 3 the traditional definition of the barrier function, and it also suffers from the curse of dimensionality during neural network

training. Indeed, it’s a good starting point for getting into the neural barrier function, and I hope I am able to finish the dynamic and implement the neural controller on the TORCS since it looks awesome and fun. I am excited to be able to get in touch with more RL relative projects like this and be able to implement them in the real car/robot in real lifes.